

DP

Aliens trick

The Type of Problem It Solves

Alien's Trick is specifically designed to solve optimization problems that have **all three** of the following characteristics:

1. **Optimization:** You need to find the maximum or minimum possible score, cost, or profit.
2. **The "Exactly K " Constraint:** You are forced to choose exactly K items, perform exactly K operations, or divide an array into exactly K segments.
3. **Convexity / Concavity (Diminishing Returns):** If you were to plot the optimal answer for $k = 1, k = 2, k = 3$, etc., the resulting graph would be a convex or concave curve. This usually means the problem has "diminishing returns" (e.g., the first segment you pick gives you a huge profit, the second gives you a bit less, the third even less, and so on).

The Aliens Trick: TL;DR Summary

1. **Drop the K Limit:** Ignore the rule that says you must use exactly K items/segments.
2. **Add a Penalty:** Pretend you can use infinite items, but you must pay a fixed penalty (λ) for every single one you use.
3. **Solve the Easy Version:** Write a fast DP to find the maximum score minus the penalties used.
4. **Binary Search the Penalty:** Keep adjusting λ until your DP naturally decides that K items is the perfect amount to use.
5. **Refund the Cost:** Take your final DP score and add back $\lambda * K$ to get your real answer.

The 3 Conditions to Use It (Cheat-Sheet)

Check these three boxes before you try to use the trick:

- **The K Bottleneck:** The problem asks you to optimize something using exactly/at most K operations, segments, or edges.
- **Easy Without K :** If the K constraint didn't exist, you could solve the problem easily with a standard greedy or 1D DP approach.
- **Diminishing Returns (Concavity):** The first item you pick gives you the biggest boost. The second gives a little less. The third gives even less. Because the benefits drop off, a uniform penalty will perfectly control how many items you take.

```

int const N = 3e5 + 5;
pair<int, int> dp[N][2];
int vis[N][2], id;
int n, k, a[N];
int lamda;
pair<int, int> rec(int i, bool lst) {
    if (i == n) return {0, 0};
    auto &ret = dp[i][lst];
    if (vis[i][lst] == id) return ret;
    vis[i][lst] = id;
    auto leave = rec(i + 1, 0);
    auto take = rec(i + 1, 1);
    take.first += a[i];
    if (!lst) {
        take.first -= lamda;
        take.second--;
    }

    return ret = max(leave, take);
}

void solve() {
    cin >> n >> k;

    for (int i = 0; i < n; ++i) cin >> a[i];

    // st = 0      "AT MOST K" (empty allowed).
    // st = -1e15   "EXACTLY K" (must force bad choices).
    // en = 1e15    Always. It forces the DP to pick 0 items.

    int st = 0, en = 1e15, md;
    __int128_t ans = 0;
    while (st <= en) {
        md = st + (en - st) / 2;
        lamda = md;

        ++id;
        auto [cur_ans, cnt] = rec(0, 0);

        int used = -cnt; // cnt that you take (neg in dp)

        if (used > k) {
            // We used too many items. The penalty is too weak.
            st = md + 1;
        } else {
            // We used <= k items. This is a valid configuration!

```

```

        // Refund the penalty for exactly K items to get the true score.
        ans = cur_ans + (__int128_t)md * k;

        // Try to find a smaller valid penalty to handle collinear points
        en = md - 1;
    }
}

cout << (int)ans << '\n';
}

```

Basics

LIS

```

int lengthOfLIS(ll nums[],int n)
{
    vector<ll> ans;
    ans.push_back(nums[0]);
    for (int i = 1; i < n; i++) {
        if (nums[i] > ans.back()) {
            ans.push_back(nums[i]);
        }
        else {
            int low = lower_bound(ans.begin(), ans.end(),nums[i])-
ans.begin();
            ans[low] = nums[i];
        }
    }
    return ans.size();
}

```

```

```cpp
for (int i = 0; i < k; i++) {
 auto [x,y] = a[i];
 int pos = upper_bound(lis.begin(), lis.end(), y) - lis.begin();
 if (pos == lis.size()) {
 lis.push_back(y);
 build.push_back(i);
 } else {
 lis[pos] = y;
 build[pos] = i;
 }
}

```

```

 dp[i] = pos + 1;
 if (pos == 0) par[i] = -1;
 else par[i] = build[pos - 1];
 }
 int maxi = lis.size();
 int lst = -1;
 for (int i = 0; i < k; i++) {
 if (dp[i] == maxi) lst = i;
 }
 vector<int> path;
 while (lst != -1) {
 path.emplace_back(lst);
 lst = par[lst];
 }

```

## Digit DP

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;
using namespace std;

#define double long double
#define int long long
#define ll int
#define MX LLONG_MAX
#define MN LLONG_MIN
#define endl '\n'
#define ordered_set tree<ll, null_type, greater<ll>, rb_tree_tag,
tree_order_statistics_node_update>
#define wady ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);

int dp[20][2][2][95][95];
string l,r;
int k;
int rec(int idx,int lower ,int upper ,int rem ,int numrem){
 if(idx == r.size())return !(rem % k) and !(numrem);
 int &ret = dp[idx][lower][upper][rem][numrem];
 if(~ret)return ret;
 ret = 0;
 for(int digit = 0;digit < 10;digit++){

```

```

 if(!upper and digit > r[idx]-'0')continue;
 if(!lower and digit < l[idx]-'0')continue;
 ret += rec(idx+1,lower | (digit > l[idx]-'0'),upper | (digit < r[idx]-
'0'),rem + digit,(numrem*10+digit)%k);
 }
 return ret;
}

void solve(int testcase) {
 cout << "Case " << testcase << ": " ;
 memset(dp,-1,sizeof dp);
 cin >> l >> r >> k;
 while(l.size() < r.size())l = '0'+l;
 if(k > 90)return cout << 0 << endl,void();
 cout << rec(0,0,00,00,0) << endl;
}

signed main() {
 wady
 int tt = 1;
#ifdef ONLINE_JUDGE
 freopen("in.txt", "r", stdin);
 freopen("out.txt", "w", stdout);
#endif
 cin >> tt;
 int k = 1;
 while (tt--) solve(k++);
}

```

## CHT

$$[i] = \min_i (k_i x + m_i)$$

```

// Description: Container where you can add lines of the form kx+m, and query
maximum values at points x.
// to find the minimum
// add(-k, -m)
// query(x) * -1

```

```

struct Line {
 ll k, m;
 mutable ll p;
 bool operator<(const Line& o) const {
 return k < o.k;
 }
}

```

```

 }
 bool operator<(const ll&x) const {
 return p < x;
 }
};

struct LineContainer : multiset<Line, less<>> {
 // (for doubles, use inf = 1/.0, div(a,b) = a/b)
 const ll inf = numeric_limits<ll>::max();
 ll div(ll a, ll b) { // floored division
 return a / b - ((a ^ b) < 0 && a % b);
 }
 bool isect(iterator x, iterator y) {
 if (y == end()) { x->p = inf; return false; }
 if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
 else x->p = div(y->m - x->m, x->k - y->k);
 return x->p >= y->p;
 }
 void add(ll k, ll m) {
 k *= -1;
 m *= -1;
 auto z = insert({k, m, 0}), y = z++, x = y;
 while (isect(y, z)) z = erase(z);
 if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
 while ((y = x) != begin() && (--x)->p >= y->p)
 isect(x, erase(y));
 }
 ll query(ll x) {
 assert(!empty());
 auto l = *lower_bound(x);
 return -(l.k * x + l.m);
 }
};

```

## Combining subtrees

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;
#define all(v) v.begin(), v.end()
template<typename T>

```

```

using orderedset = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
#define ll long long
#define int ll
#define ld double
#define endl '\n'
#define wady \
 ios_base::sync_with_stdio(0); \
 cin.tie(0); \
 cout.tie(0);

void files() {
#ifdef ONLINE_JUDGE
 freopen("in.txt", "r", stdin);
 freopen("out.txt", "w", stdout);
#endif
}

#pragma GCC optimize("O3", "unroll-loops")
#pragma GCC target("avx2", "popcnt")

ll const inf = 1e18;

int const N = 1005;
ll a[N];
vector<int> adj[N];
vector<ll> dp[N];
int n;

vector<int> merge(const vector<int> &dp_root, const vector<int> &dp_child) {
 int n = dp_root.size(), m = dp_child.size();
 vector<int> ret(n + m - 1, -1e18);
 ret[0] = 0;
 // i = 1, MUST take the root
 for (int i = 1; i < n; ++i)
 for (int j = 0; j < m; ++j)
 ret[i + j] = max<ll>(ret[i + j], 0LL + dp_root[i] + dp_child[j]);
 return ret;
}

void dfs(int u, int p) {
 dp[u] = {0, a[u]};
 for (auto &v: adj[u]) {
 if (v == p) continue;
 dfs(v, u);
 dp[u] = merge(dp[u], dp[v]);
 }
}

```

```

 }
}

void solve(int tc) {
 cin >> n;
 for (int i = 1; i <= n; ++i) cin >> a[i];
 for (int i = 1; i < n; ++i) {
 int u, v;
 cin >> u >> v;
 adj[u].push_back(v);
 adj[v].push_back(u);
 }

 dfs(1, -1);

 vector<int> val(n + 1, -1e18);
 for (int i = 1; i <= n; i++) {
 int sz = dp[i].size();
 for (int j = 1; j < sz; j++) {
 val[j] = max(val[j], dp[i][j]);
 }
 }

 ll mx[n + 1][n + 1]{};
 for (int i = 1; i <= n; ++i) {
 ll cur = -1e18;
 for (int j = i; j <= n; ++j) {
 cur = max(cur, val[j]);
 mx[i][j] = cur;
 }
 }

 /*for (int i = 1; i <= n; ++i) cout << dp[1][i] << ' ';
 cout << '\n';
 */

 int q;
 cin >> q;
 while (q--) {
 int l, r;
 cin >> l >> r;
 cout << mx[l][r] << '\n';
 }
}

signed main() {
 wady

```



```

files();
int t = 1;
int tc = 1;
// cin >> t;
while (t--)
 solve(tc++);
}

```

## Divide and Conquer

$$dp(i, j) = \min_{0 \leq k \leq j} dp(i-1, k-1) + C(k, j)$$

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;
template<typename T>
using orderedset = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
#define int long long
#define ll long long
#define ld long double
#define endl '\n'
#define wady \
ios_base::sync_with_stdio(0); \
cin.tie(0); \
cout.tie(0);

void files() {
#ifdef ONLINE_JUDGE
 freopen("in.txt", "r", stdin);
 freopen("out.txt", "w", stdout);
#endif
}

`const int N = 1e5 + 20, K = 22;
int dp[K][N];
int a[N];
int ans = 0;
int frq[N];

void add(int idx) {

```

```

 int cont = (frq[a[idx]] * (frq[a[idx]] - 1)) / 2;
 ans -= cont;
 frq[a[idx]]++;
 cont = (frq[a[idx]] * (frq[a[idx]] - 1)) / 2;
 ans += cont;
 }

 void rem(int idx) {
 int cont = (frq[a[idx]] * (frq[a[idx]] - 1)) / 2;
 ans -= cont;
 frq[a[idx]]--;
 cont = (frq[a[idx]] * (frq[a[idx]] - 1)) / 2;
 ans += cont;
 }

 int n, k;
 int ql = 1, qr = 0;

 int query(int l, int r) {
 while (ql > l) add(--ql);
 while (qr < r) add(++qr);
 while (ql < l) rem(ql++);
 while (qr > r) rem(qr--);
 return ans;
 }

 void rec(int i, int l, int r, int optl, int optr) {
 if (l > r) return;
 int mid = (l + r) >> 1;
 int best = optl;
 int mini = 1e16;
 for (int j = optl; j <= min(mid - 1, optr); j++) {
 int cost = query(j + 1, mid);
 int val = dp[i - 1][j] + cost;
 if (val < mini) {
 mini = val;
 best = j;
 }
 }
 dp[i][mid] = mini;
 rec(i, l, mid - 1, optl, best);
 rec(i, mid + 1, r, best, optr);
 }

 void solve(int tc) {
 cin >> n >> k;
 }

```

```

 for (int i = 1; i <= n; i++) cin >> a[i];
 for (int j = 0; j <= n; j++) dp[0][j] = 1e16;
 dp[0][0] = 0;
 for (int i = 1; i <= k; i++) rec(i, 1, n, 0, n - 1);
 cout << dp[k][n] << endl;
}

```

```

signed main() {
 wady
 files();
 int t = 1;
 int tc = 1;
 while (t--)
 solve(tc++);
}

```

## Cp algo generic implementation

```

int m, n;
vector<long long> dp_before, dp_cur;

long long C(int i, int j);

// compute dp_cur[l], ... dp_cur[r] (inclusive)
void compute(int l, int r, int optl, int optr) {
 if (l > r)
 return;

 int mid = (l + r) >> 1;
 pair<long long, int> best = {LLONG_MAX, -1};

 for (int k = optl; k <= min(mid, optr); k++) {
 best = min(best, {(k ? dp_before[k - 1] : 0) + C(k, mid), k});
 }

 dp_cur[mid] = best.first;
 int opt = best.second;

 compute(l, mid - 1, optl, opt);
 compute(mid + 1, r, opt, optr);
}

long long solve() {
 dp_before.assign(n, 0);
}

```

```

dp_cur.assign(n, 0);

for (int i = 0; i < n; i++)
 dp_before[i] = C(0, i);

for (int i = 1; i < m; i++) {
 compute(0, n - 1, 0, n - 1);
 dp_before = dp_cur;
}

return dp_before[n - 1];
}

```

## Knuth

$$dp(i, j) = \min_{0 \leq k \leq j} dp(i - 1, k - 1) + C(k, j)$$

## Cp algo

```

int solve() {
 int N;
 ... // read N and input
 int dp[N][N], opt[N][N];

 auto C = [&](int i, int j) {
 ... // Implement cost function C.
 };

 for (int i = 0; i < N; i++) {
 opt[i][i] = i;
 ... // Initialize dp[i][i] according to the problem
 }

 for (int i = N-2; i >= 0; i--) {
 for (int j = i+1; j < N; j++) {
 int mn = INT_MAX;
 int cost = C(i, j);
 for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k++) {
 if (mn >= dp[i][k] + dp[k+1][j] + cost) {
 opt[i][j] = k;
 mn = dp[i][k] + dp[k+1][j] + cost;
 }
 }
 dp[i][j] = mn;
 }
 }
}

```

```

 }

 return dp[0][N-1];
}

```

```

const int N = 5000 + 20;
int a[N];
int dp[N][N], opt[N][N];
int n;

void solve(int tc) {
 cin >> n;
 for (int i = 1; i <= n; i++) {
 cin >> a[i];
 a[i] += a[i - 1];
 }
 auto cost = [&](int i, int j) {
 return a[j] - a[i - 1];
 };
 for (int i = 0; i <= n; i++) {
 opt[i][i] = i;
 }
 for (int i = n - 1; i > 0; i--) {
 for (int j = i + 1; j <= n; j++) {
 int mini = 1e18;
 int C = cost(i, j);
 for (int k = opt[i][j - 1]; k <= min(j - 1, opt[i + 1][j]); k++) {
 if (mini >= dp[i][k] + dp[k + 1][j] + C) {
 opt[i][j] = k;
 mini = dp[i][k] + dp[k + 1][j] + C;
 }
 }
 dp[i][j] = mini;
 }
 }
 cout << dp[1][n] << endl;
}

```

## Li chao Tree

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;

```

```

const ll inf = 2e18;

struct Line {
 ll m, c;
 ll eval(ll x) {
 return m * x + c;
 }
};

struct node {
 Line line;
 node* left = nullptr;
 node* right = nullptr;
 node(Line line) : line(line) {}
 void add_segment(Line nw, int l, int r, int L, int R) {
 if (l > r || r < L || l > R) return;
 int m = (l + 1 == r ? l : (l + r) / 2);
 if (l >= L and r <= R) {
 bool lef = nw.eval(l) < line.eval(l);
 bool mid = nw.eval(m) < line.eval(m);
 if (mid) swap(line, nw);
 if (l == r) return;
 if (lef != mid) {
 if (left == nullptr) left = new node(nw);
 else left -> add_segment(nw, l, m, L, R);
 }
 else {
 if (right == nullptr) right = new node(nw);
 else right -> add_segment(nw, m + 1, r, L, R);
 }
 }
 return;
 }
 if (max(l, L) <= min(m, R)) {
 if (left == nullptr) left = new node({0, inf});
 left -> add_segment(nw, l, m, L, R);
 }
 if (max(m + 1, L) <= min(r, R)) {
 if (right == nullptr) right = new node({0, inf});
 right -> add_segment(nw, m + 1, r, L, R);
 }
}

ll query_segment(ll x, int l, int r, int L, int R) {
 if (l > r || r < L || l > R) return inf;
 int m = (l + 1 == r ? l : (l + r) / 2);
 if (l >= L and r <= R) {
 ll ans = line.eval(x);
 if (l < r) {

```

```

 if (x <= m && left != nullptr) ans = min(ans, left -> query_segment(x,
l, m, L, R));
 if (x > m && right != nullptr) ans = min(ans, right ->
query_segment(x, m + 1, r, L, R));
 }
 return ans;
}
ll ans = inf;
if (max(l, L) <= min(m, R)) {
 if (left == nullptr) left = new node({0, inf});
 ans = min(ans, left -> query_segment(x, l, m, L, R));
}
if (max(m + 1, L) <= min(r, R)) {
 if (right == nullptr) right = new node ({0, inf});
 ans = min(ans, right -> query_segment(x, m + 1, r, L, R));
}
return ans;
}
};

```

```

struct LiChaoTree {
 int L, R;
 node* root;
 LiChaoTree() : L(numeric_limits<int>::min() / 2),
R(numeric_limits<int>::max() / 2), root(nullptr) {}
 LiChaoTree(int L, int R) : L(L), R(R) {
 root = new node({0, inf});
 }
 void add_line(Line line) {
 root -> add_segment(line, L, R, L, R);
 }
 // y = mx + b: x in [l, r]
 void add_segment(Line line, int l, int r) {
 root -> add_segment(line, L, R, l, r);
 }
 ll query(ll x) {
 return root -> query_segment(x, L, R, L, R);
 }
 ll query_segment(ll x, int l, int r) {
 return root -> query_segment(x, l, r, L, R);
 }
};

```

```

int32_t main() {
 ios_base::sync_with_stdio(0);
 cin.tie(0);

```

```

LiChaoTree t = LiChaoTree((int)-1e9, (int) 1e9);
int n, q; cin >> n >> q;
for (int i = 0; i < n; i++) {
 ll l, r, a, b; cin >> l >> r >> a >> b;
 r--;
 t.add_segment({a, b}, l, r);
}
while (q--) {
 int ty; cin >> ty;
 if (ty == 0) {
 ll l, r, a, b; cin >> l >> r >> a >> b;
 r--;
 t.add_segment({a, b}, l, r);
 }
 else {
 ll x; cin >> x;
 ll ans = t.query(x);
 if (ans >= inf) cout << "INFINITY\n";
 else cout << ans << '\n';
 }
}
return 0;
}
// https://judge.yosupo.jp/problem/segment_add_get_min

```

## SOS and profiles

### SOS DP

```

// sum over subsets
for (int i = 0; i < B; i++) {
 for (int mask = 0; mask < (1 << B); mask++) {
 if ((mask & (1 << i)) != 0) {
 f[mask] += f[mask ^ (1 << i)];
 }
 }
}

// sum over supersets
for (int i = 0; i < B; i++) {
 for (int mask = (1 << B) - 1; mask >= 0; mask--) {
 if ((mask & (1 << i)) == 0) g[mask] += g[mask ^ (1 << i)];
 }
}

```



## DP on profiles

```
// sum over subsets
for (int i = 0; i < B; i++) {
 for (int mask = 0; mask < (1 << B); mask++) {
 if ((mask & (1 << i)) != 0) {
 f[mask] += f[mask ^ (1 << i)];
 }
 }
}

// sum over supersets
for (int i = 0; i < B; i++) {
 for (int mask = (1 << B) - 1; mask >= 0; mask--) {
 if ((mask & (1 << i)) == 0) g[mask] += g[mask ^ (1 << i)];
 }
}
```